

Reviewer Pack — Log-Size Submodularity in Monotone DNF for k-CLIQUE

Entry 2222bcac7069 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 03:56:45 UTC

Log-Size Submodularity in Monotone DNF for k-CLIQUE

Entry ID: 2222bcac7069 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 03:56:45 UTC

1. Conjecture

Field A (mathematical branch): BOOLEAN_FUNCTION_APPROXIMATION **Field B** (complexity object): MONOTONE_CIRCUIT_COMPLEXITY

Statement:

For any monotone DNF formulas A and B, the logarithm of the number of terms in $A \wedge B$ is at most the sum of the logarithms of the number of terms in A and B. Furthermore, any monotone DNF formula representing the k-CLIQUE function has $\Omega(n)$ log-size, while poly-sized DNF formulas have $O(\log n)$ log-size.

Rationale (proposer’s reasoning):

The submodularity of log-size under conjunction implies that combining DNFs doesn’t exponentially increase the term count, which could be leveraged to show lower bounds. The $\Omega(n)$ bound for k-CLIQUE suggests that efficient DNF representations are impossible, aligning with known lower bounds.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a4768b12614dbc2f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        edges = set()
        for i in range(k):
            for j in range(i + 1, k):
                edges.add((i, j))
        for _ in range(2 * k, n):
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            if (u, v) not in edges and (v, u) not in edges:
                edges.add((u, v))
        return edges

    def dnf_size(edges):
        n = max(max(u, v) for u, v in edges) + 1
        clauses = []
        for i in range(n):
            clause = [j for j in range(n) if (i, j) not in edges]
            if clause:
                clauses.append(clause)
        return len(clauses)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            edges = generate_k_clique(n, random.randint(2, min(5, n // 2)))
            dnf_size_val = dnf_size(edges)
            results.append(dnf_size_val)

    mean_dnf_size = sum(results) / len(results)
    if mean_dnf_size < n:
        return {
            "metric_name": "mean_dnf_size",
            "metric_value": mean_dnf_size,
```

```

        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "mean_dnf_size is too small"
    }
else:
    return {
        "metric_name": "mean_dnf_size",
        "metric_value": mean_dnf_size,
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_dnf_size = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_dnf_size} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_dnf_size} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mean_dnf_size is too small' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ed': 30, 'conjecture_holds': False, 'counterexample': 'mean_dnf_size is too small'}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.2, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.433333333333334, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 18.9, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.366666666666667, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.166666666666668, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 18.966666666666665, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.3, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'mean_dnf_size', 'metric_value': 19.133333333333333, 'instances_tested': 30, 'conjecture_holds': True, 'counterexample': ''}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43ebdb7e.py", line 85, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_43ebdb7e.py", line 85, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

~^~^~^~^~^~^

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing validation; counterexample metrics may be unreliable |
next: Re-run test with seed tracking enabled and error handling for metric collection

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	90320
2	propose	ollama_remote	qwen3:8b	0	0	39495
3	propose	ollama_remote	qwen3:8b	0	0	59947
4	propose	ollama_remote	qwen3:8b	0	0	117200
5	propose	ollama_remote	qwen3:8b	0	0	98920
6	preregistration	ollama_remote	qwen3:8b	0	0	29719
7	novelty	ollama_remote	qwen3:8b	0	0	24042
8	novelty	ollama_remote	qwen3:8b	0	0	15781
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12008
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8672
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12454
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8921
13	judge	ollama_remote	qwen3:8b	0	0	22665

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 540144 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in `research/audit/2222bcac7069.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2222bcac7069.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2222bcac7069.tar.gz` (if generated)